



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Моделирование сред и разработка приложений виртуальной
и дополненной реальности»

Практическая работа № 5

Студент группы *ИКБО-50-23, Враженко Д.О.*

(подпись)

Старший
преподаватель *Горбатов Г.В.*

(подпись)

Отчет представлен «__» _____ 202__ г.

Москва 2026 г.

1. Создание PTP соединение между клиентами (интеграция внешних систем)

В ходе выполнения практической работы была реализована многопользовательская система взаимодействия между клиентами с использованием Unity Netcode for GameObjects и Unity Transport.

Было создано peer-to-peer подключение между участниками игровой сессии. Один пользователь выступает в роли Host (сервер + клиент), остальные подключаются как Client.

Для подключения использовались встроенные средства Unity Networking:

- NetworkManager;
- Unity Transport;
- Netcode for GameObjects.

Также был реализован пользовательский интерфейс с кнопками:

- Host;
- Client;
- XR Mode;
- Desktop Mode.

После подключения кнопки Host и Client автоматически скрываются.

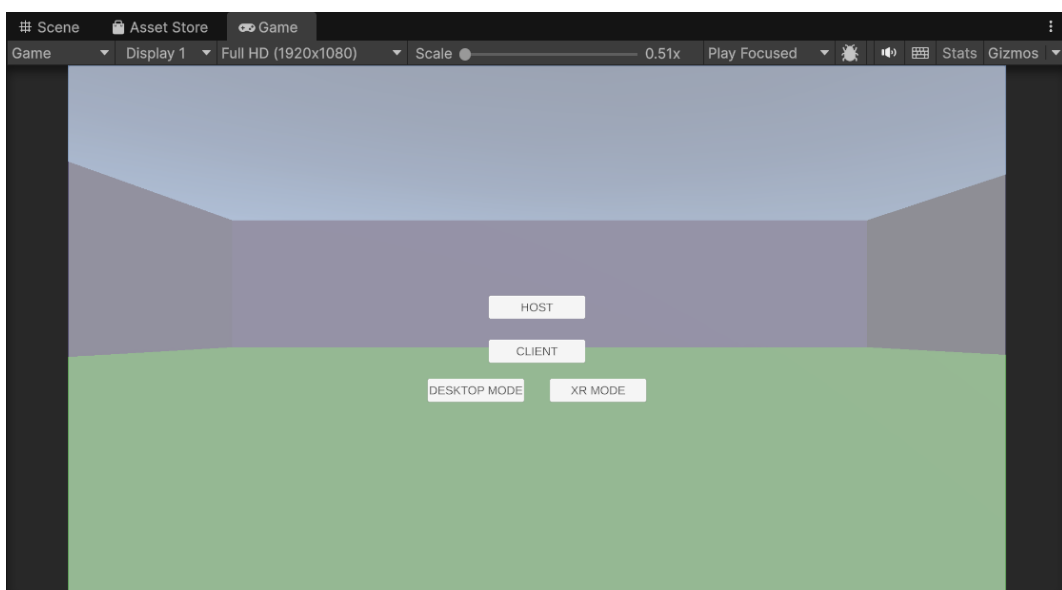


Рисунок 1 — Интерфейс подключения к игровой сессии

Листинг 1 - Скрипт создания сетевого подключения

```
using Unity.Netcode;
using UnityEngine;

public class SimpleConnection : MonoBehaviour
{
    public GameObject connectionPanel;

    public void StartHost()
    {
        NetworkManager.Singleton.StartHost();
        connectionPanel.SetActive(false);
        Debug.Log("HOST STARTED");
    }

    public void StartClient()
    {
        NetworkManager.Singleton.StartClient();
        connectionPanel.SetActive(false);
        Debug.Log("CLIENT STARTED");
    }
}
```

2. Получение и обработка пользовательского пакета данных PTS (на основе встроенных решений)

Для передачи пользовательских данных использовалась встроенная система синхронизации Unity Netcode.

Каждый игрок представляет собой сетевой объект NetworkObject, который автоматически синхронизируется между всеми клиентами.

Были реализованы:

- синхронизация позиции игроков;
- передача пользовательского состояния;
- обработка локального ввода.

Для ограничения количества участников был установлен лимит до 30 игроков одновременно.

Каждому подключенному игроку автоматически назначался уникальный цвет игрового объекта.

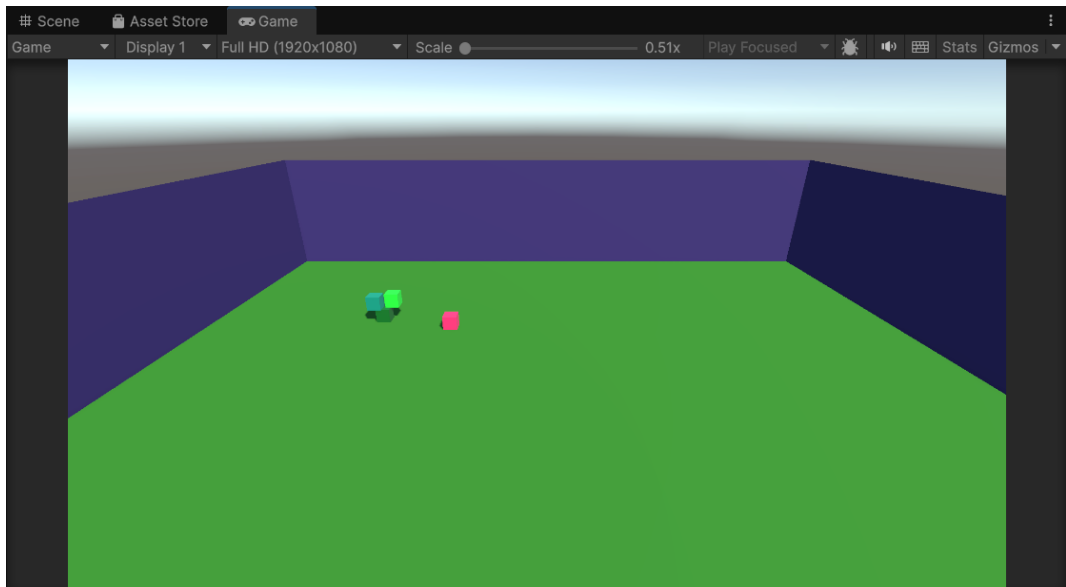


Рисунок 2 — Подключение нескольких игроков к серверу

Движение игрока реализовано с помощью:

- клавиш WASD;
- прыжка на Space;
- Rigidbody.

Обработка пользовательского ввода выполняется только для локального игрока через проверку IsOwner.

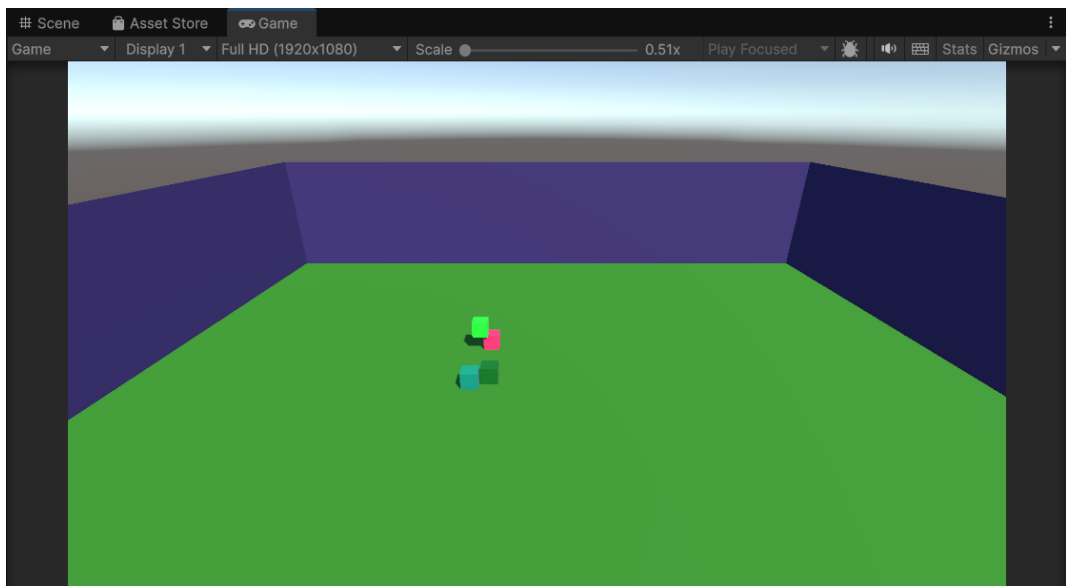


Рисунок 3 — Управление игровым персонажем

Листинг 2 - Скрипт управления сетевым игроком

```
using Unity.Netcode;
using UnityEngine;

public class PlayerController : NetworkBehaviour
```

```

{
    [Header("Movement")]
    public float speed = 5f;
    public float jumpForce = 5f;

    [Header("Components")]
    public Rigidbody rb;
    public MeshRenderer meshRenderer;

    private bool isGrounded;
    public NetworkVariable<Color> playerColor =
        new NetworkVariable<Color>();

    void Start()
    {
        playerColor.OnValueChanged += OnColorChanged;
        ApplyColor(playerColor.Value);
    }

    public override void OnNetworkSpawn()
    {
        if (IsOwner)
        {
            Color randomColor = new Color(Random.value, Random.value,
Random.value);
            SubmitColorServerRpc(randomColor);
        }
    }

    void Update()
    {
        if (!IsOwner) return;
        Move();
        Jump();
    }

    void Move()
    {
        float h = Input.GetAxis("Horizontal");
        float v = Input.GetAxis("Vertical");
        Vector3 move = new Vector3(h, 0, v);
        transform.position += move * speed * Time.deltaTime;
    }

    void Jump()
    {
        if (Input.GetKeyDown(KeyCode.Space) && isGrounded)
            rb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);
    }

    private void OnCollisionEnter(Collision collision)
    {
        if (collision.gameObject.CompareTag("Ground"))
            isGrounded = true;
    }

    private void OnCollisionExit(Collision collision)
    {
        if (collision.gameObject.CompareTag("Ground"))
            isGrounded = false;
    }

    [ServerRpc]

```

```

void SubmitColorServerRpc(Color color)
{
    playerColor.Value = color;
}

void OnColorChanged(Color oldColor, Color newColor)
{
    ApplyColor(newColor);
}

void ApplyColor(Color color)
{
    meshRenderer.material.color = color;
}
}

```

3. XR сборка проекта (применение и адаптация VR, MR или AR контроллеров)

В проект была интегрирована XR-подсистема Unity OpenXR.

Была реализована возможность переключения между:

- обычным Desktop режимом;
- XR режимом.

Для XR использовались:

- XR Origin;
- OpenXR Plugin;
- XR Interaction Toolkit.

XR-камера была размещена рядом с игровой ареной для корректного отображения сцены после запуска приложения.

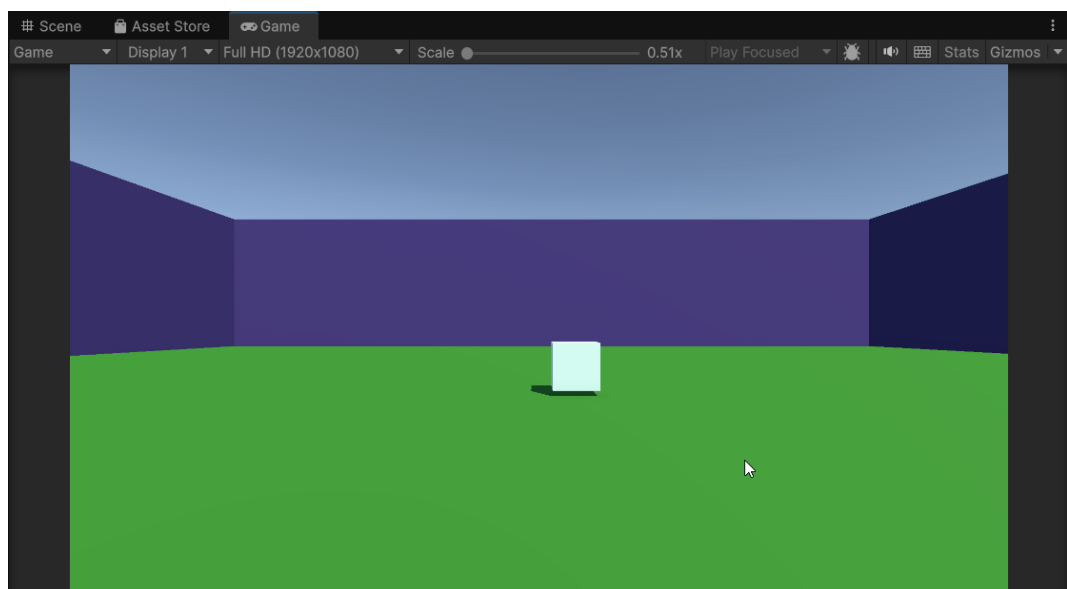


Рисунок 4 — Работа проекта в XR режиме

Листинг 3 - Скрипт переключения XR режима

```
using UnityEngine;

public class ModeSwitcher : MonoBehaviour
{
    public GameObject desktopCamera;
    public GameObject XROrigin;

    void Start()
    {
        EnableDesktopMode();
    }

    public void EnableDesktopMode()
    {
        desktopCamera.SetActive(true);
        XROrigin.SetActive(false);
        Debug.Log("Desktop mode enabled");
    }

    public void EnableXRMode()
    {
        desktopCamera.SetActive(false);
        XROrigin.SetActive(true);
        Debug.Log("XR mode enabled");
    }
}
```